



**LANDMARK METROPOLITAN
UNIVERSITY INSTITUTE**
THE PRIDE OF AFRICA

DATABASE MANAGEMENT & SQL

LANDMARK DIGITAL LESSONS

(+237) 672 339 570 / 694 990 622 | www.landmark.cm

Miss Precious Oben,
2025/2026



OUTLINE



- Introduction to Databases
- Components of DBMS
- Evolution of Databases
- Database architecture
- Introduction to SQL
- Activity



INTRODUCTION TO DATABASES



- Every day, individuals and organizations generate huge amounts of data. For this data to be useful, it must be stored in an organized way so it can be found and used easily.
- **A database is an organized collection of data.**
- There are two main types of databases:
- **Relational (SQL)** - stores data in tables (rows and columns)
- **Non-Relational (NoSQL)** - stores data in documents, key-value pairs, or graphs
- Generally, there are two types of databases; Relational and Non-relational.

INTRODUCTION TO DATABASES



- Relational Databases store data in a tabular format while in non-relational databases, data is stored in key - value pairs, graphs or documents.

Table: customers

customer_id	first_name	last_name	phone	country
1	John	Doe	817-646-8833	USA
2	Robert	Luna	412-862-0502	USA
3	David	Robinson	208-340-7906	UK
4	John	Reinhardt	307-242-6285	UK
5	Betty	Taylor	806-749-2958	UAE

Relational Databases Vs Non-Relational Databases

Elements of a database	Relational	Non-Relational
Type of data	Data is structured	Data is non-structured
Data Structure	Tabular structure	Various models but not tabular
Scalability	Limited scalability	High scalability
Schema	Predefined schema	Can accommodate various types of data
Development	Requires more development effort	Easier to develop and requires fewer resources.
Query Language	Uses SQL for querying and manipulating data	Uses their own query language or APIs

INTRODUCTION TO DATABASES



Relational Databases are also known as SQL Databases while Non-relational Databases are known as NoSQL Databases.

Use Cases For Relational Databases

Best for structured data needing complex queries. Examples: banking systems, inventory management, CRM.

Use Cases For Non-Relational Databases

Best for flexible, scalable data. Examples: social media, gaming apps, search engines.

INTRODUCTION TO DATABASES



- Examples of Relational Databases are databases generated by MySQL, Microsoft SQL Server, Oracle Database and PostgreSQL.
- Examples of NoSQL Databases are databases generated by MongoDB, Redis, Cassandra

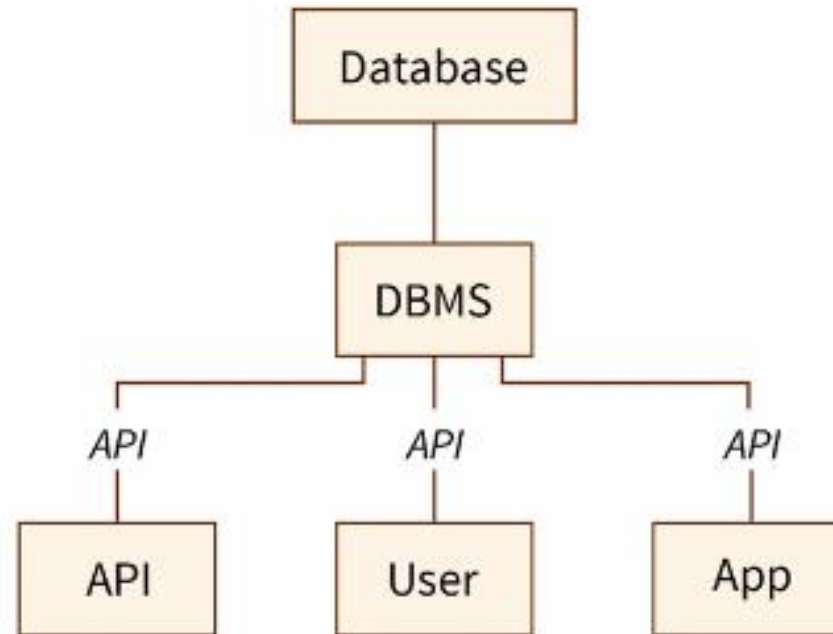
Databases are managed by **Database Management Systems (DBMS)**. As there are Relational and Non-Relational Databases so are there Non-Relational Database Management Systems and Relational Database Management Systems.

INTRODUCTION TO DATABASES



- A Database Management System (DBMS) is software that manages a database. It stores and organizes data from many sources. Examples: MySQL, Oracle Database, MongoDB.
- Recall RDBMS; In this software, data is stored in the format of tables.
- To understand how a DBMS works, we need to look at its key components.

COMPONENTS OF A DBMS



COMPONENTS OF A DBMS



- **Users:** There can be multiple users, like someone who manages the database (the database administrator), system developers, and also those who are just regular users like the customer.
- **Application:** The application of a database can be either departmental or personal or may be for internal use in an organization.
- **DBMS:** This is software that helps the users create the database and access the data inside it in an efficient manner.
- **Database:** Which is a collection of data stored in the form of a single unit.

COMPONENTS OF A DBMS



Other Important components of a DBMS are:

- **Storage engine.** Stores all the data. It works with the operating system to read and write data to disk.
- **Metadata catalog.** Also called a data dictionary. Keeps a record of all database objects (tables, schemas, users). The DBMS uses it to verify user requests.
- **Database access language:** SQL is the main database access language. It includes DDL (define structures), DML (read/modify data), TCL (manage transactions), and DCL (control access).
- **Optimization engine:** Converts your query into efficient commands the database can execute quickly.
- **Query processor:** Runs the optimized query and returns the results to the user.

EVOLUTION OF DATABASES



Before the evolution of technology which led to the conception of the modern database systems, databases have existed for centuries in different forms. The maintenance of records and data has evolved from engravings to cards to digital storage.

1. File-Based Systems (Pre-1960s): Data was managed through file based systems, where data was stored in flat files. Each file was essentially a collection of records, and programs were written specifically to manipulate these files. This approach had significant limitations in terms of data redundancy, consistency, and data sharing.

2. Hierarchical and Network Databases (1960s-1970s)

- **Hierarchical Databases:** Introduced by IBM, hierarchical databases (like IMS) organized data in a tree-like structure, where a parent-child relationship existed between data entities. This model was efficient for certain types of transactions but lacked flexibility.
- **Network Databases:** Developed to address some limitations of the hierarchical model, network databases (like IDMS) allowed more complex relationships by permitting many-to-many relationships among data. This was achieved through a network or graph structure.

3. Relational Databases (1970s-Present)

- The concept of relational databases was introduced by E.F. Codd in 1970, revolutionizing data management. This model used tables to store data and SQL for manipulation.
- In the 1980s, relational database management systems (RDBMS) like Oracle, MySQL, and Microsoft SQL Server became commercialized, offering improved data integrity, flexibility, and the ability to manage vast amounts of data across many industries.

4. NoSQL and NewSQL Databases (2000s-Present)

- **NoSQL Databases:** Emerged to address the scalability and flexibility needs of modern applications, especially with the rise of big data and real-time web applications.
- **NewSQL Databases:** Seek to combine the scalability of NoSQL systems with the ACID (Atomicity, Consistency, Isolation, Durability) guarantees of traditional RDBMS. They are designed to handle high transaction rates and complex query processing e.g. Google Spanner



5. Cloud-Based Databases and Database as a Service (DBaaS) (2010s-Present)

- **Cloud-Based Databases:** The advent of cloud computing has led to the development of cloud-based databases, which offer scalability, flexibility, and cost-effectiveness. Services like Amazon RDS, Google Cloud SQL, and Microsoft Azure SQL Database provide managed database services in the cloud.
- **Database as a Service (DBaaS):** This model allows users to access and use database functionality without the need to install, operate, and maintain the database system themselves. It emphasizes ease of use, scalability, and on-demand access.

DATABASE ARCHITECTURES



Database architecture describes how a database system is designed and organized. It covers how data is stored, accessed, and managed across hardware, software, and processes. Good architecture ensures performance, security, and scalability.

Key components: Database model, Storage Engine, Database access language, Query Processor and Transaction Management.

DATABASE ARCHITECTURES



- **Single-Tier Architecture:** All components (database, DBMS, app) run on one machine.
- **Two-Tier Architecture:** The app (client) and the database (server) are on separate machines.
- **Three-Tier Architecture:** Adds a middle layer (app/web server) between the client and database. More scalable and secure.
- **N-Tier Architecture:** Extends three-tier with more layers (caching, load balancing, etc.). Very scalable and flexible.

DATABASE ARCHITECTURES



- **Distributed Database Architecture:** Involves spreading the database across multiple sites, servers, or locations. It's designed for redundancy, high availability, and improved performance, especially in global applications.

Modern Considerations in Database Architecture

- **Cloud-Based Databases:** Many modern databases are designed to run in cloud environments, offering scalability, managed services, and distributed computing capabilities.
- **Big Data:** Architectures now often incorporate technologies for handling large volumes of structured and unstructured data, such as Hadoop and NoSQL databases.
- **Data Security:** As part of the architecture, databases include mechanisms for ensuring data privacy, protection, and compliance with regulations.

INTRODUCTION TO SQL



- SQL “Sequel” stands for Structured Query Language.
- It is the Database Access Language for RDBMS like MySQL, PostgreSQL, Microsoft SQL Server, Oracle Database etc.
- It is used in creating databases and manipulating data in databases.

SQL is widely adopted today because:

- It is fast
- It is a portable language and
- It is a platform independent language

INTRODUCTION TO SQL



SQL works through commands grouped into three main categories: DDL, DML, and TCL.

- **DDL:** Commands used to create and define database structures.
Examples:

INTRODUCTION TO SQL



- To create, drop, and alter.
 - To grant and revoke various roles and privileges.
Examples are DROP, ALTER, CREATE and TRUNCATE.
- **DML:** Commands used to read and change data in the database.
Examples:
Examples are INSERT, DELETE, SELECT and UPDATE.
- **TCL:** Commands used to manage transactions. Examples are COMMIT, ROLLBACK and SAVEPOINT.
Examples are COMMIT, ROLLBACK and SAVE TRANSACTION.

SQL QUERIES



- A Query is a statement which declares what data we are looking for, where to find it in the database, and optionally, how to transform it before it is returned.
- Queries consist of keywords such as SELECT, FROM, WHERE
- The general syntax of a common query in SQL is:

SELECT column, another_column, ... FROM tablename;

COMMON QUERIES



- CREATE DATABASE databasename;
- CREATE TABLE employees (employee_id INT PRIMARY KEY, first_name VARCHAR(50), last_name VARCHAR(50), email VARCHAR(100), hire_date DATE, salary DECIMAL(10, 2), department_id INT);
- SHOW TABLES;
- SHOW DATABASES;
- INSERT INTO tablename(column1,column2) VALUES (value1,value2);
- SELECT * FROM tablename;
- SELECT * FROM tablename WHERE condition;
- DROP DATABASE databasename;
- DROP TABLE databasename.tablename;

DATABASE NAMING CONVENTIONS



When naming your database consider the following;

- A clear and descriptive name
- Avoid reserved words
- Consistency
- Simple and short names
- Case-sensitivity
- Portability

TABLES IN SQL



- A table is made up of rows (records) and columns (attributes/fields).
- Think of it like a real-world object. A Students table could have columns: student_id, name, age, school_id.schoolID.
- In SQL a table must have fields even if they hold no records.
- Every column must have a data type (e.g., INT, VARCHAR, DATE, DECIMAL). The data type controls what kind of data can be stored and what operations can be done on it.

TABLES IN SQL



- Columns can have constraints that control what data is allowed. Common constraints: NOT NULL (the column must always have a value), PRIMARY KEY (uniquely identifies each row), UNIQUE (no two rows can have the same value in this column).
- Every value in a column must match the column's data type. This keeps the data consistent and makes queries reliable.

ACTIVITY



- Download MySQL Server and MySQL Workbench
- Create your first database and table.

